

JavaScript

Memory Leaks & Garbage Collection

Memory Management · 5 Common Leaks · WeakRef · JS Internals · Interview Prep

MEMORY LIFE CYCLE

1. **ALLOCATE** -> JS allocates memory when you create variables/objects
2. **USE** -> Read and write values
3. **RELEASE** -> GC frees memory when no longer needed

KEY INSIGHT: GC can only free what it **KNOWS** is unreachable.

If you keep accidental references -> GC cannot free -> **MEMORY LEAK**

Garbage Collection — Mark & Sweep

JavaScript uses the **Mark and Sweep** algorithm. GC starts from the root (global scope), marks every reachable object, then sweeps (frees) everything that was not marked.

```
let user = { name: 'Akshay' }; // reachable – heap allocated

user = null; // reference cut

// GC asks: can I reach { name: 'Akshay' } from anywhere?

// Answer: NO -> mark for collection -> memory freed
```

```
ROOT (global scope)

|

+-- user --> { name: 'Akshay' } REACHABLE (kept)

|

+-- obj --> { id: 1 } REACHABLE (kept)

|

+-- (none)--> { age: 25 } UNREACHABLE (freed)
```

What is a Memory Leak?

A memory leak happens when your code holds references to objects that are no longer needed — preventing the GC from freeing them. Memory grows continuously until the app slows down or crashes.

```
Healthy memory pattern: /\ /\ /\ /\ (rises and falls with GC)

Leaking memory pattern: ////////// (only rises, never falls)
```

5 Most Common Memory Leaks

#1

Accidental Global Variables

```
function createUser() {  
  
  userName = 'Akshay'; // Missing let/const -> GLOBAL variable!  
  
}  
  
// userName attaches to window.userName  
  
// window is always reachable -> GC can NEVER collect it!  
  
// Fix 1 – always use let/const/var  
  
function createUser() {  
  
  const userName = 'Akshay'; // local, freed when fn exits  
  
}  
  
// Fix 2 – strict mode catches it immediately  
  
'use strict';  
  
userName = 'Akshay'; // ReferenceError! caught at runtime
```

#2

Forgotten Timers & Intervals

```
const user = { data: new Array(100000).fill('x') };  
  
// Interval holds reference to callback AND everything it uses  
  
const id = setInterval(() => {  
  
  console.log(user.data); // closure keeps 'user' alive!  
  
}, 1000);  
  
// If never cleared -> user object NEVER gets GC'd -> LEAK!  
  
// Fix – always clear when done  
  
clearInterval(id); // releases callback + closure references  
  
// React – clear in useEffect cleanup  
  
useEffect(() => {
```


#3

Detached DOM Nodes

```
let detachedNode;

function createLeak() {

  const div = document.createElement('div');

  document.body.appendChild(div);

  detachedNode = div; // outer ref kept!

  document.body.removeChild(div); // removed from DOM

}

// div is gone from DOM but detachedNode still points to it -> LEAK!

// Fix – null the reference when done

detachedNode = null; // GC can now collect the old node
```

#4

Event Listeners Not Removed

```
class Component {

  constructor() {

    this.data = new Array(10000).fill('heavy data');

    // closure over 'this' keeps entire Component in memory!

    document.addEventListener('click', this.handleClick);

  }

  handleClick() { console.log(this.data); }

  destroy() {

    // Must remove listener when done

    document.removeEventListener('click', this.handleClick); //

  }

}

// React fix

useEffect(() => {

  window.addEventListener('resize', handleResize);

  return () => window.removeEventListener('resize', handleResize);

}, []);
```

```
}, []);
```

#5

Closures Holding Large References

```
function createProcessor() {

  const hugeData = new Array(1000000).fill('x'); // 1MB!

  // closure holds ALL of hugeData even though only [0] is used!

  return function process() {

    return hugeData[0]; // only needs index 0 but holds 1MB!

  };

}

// Fix – extract only what you need before closing over it

function createProcessor() {

  const hugeData = new Array(1000000).fill('x');

  const firstItem = hugeData[0]; // extract what's needed

  // hugeData goes out of scope -> GC can free 1MB

  return function process() { return firstItem; }; //

}
```

WeakRef & FinalizationRegistry (ES2021)

Intermediate -> Expert

WeakRef holds a reference that does NOT prevent garbage collection. Use it when you want to cache an object but allow GC to reclaim it if memory is needed.

```
let user = { name: 'Akshay' };

const weakRef = new WeakRef(user);

user = null; // remove strong reference

// GC may now collect the original object

// Check if still alive before using

const obj = weakRef.deref();

if (obj) {

  console.log(obj.name); // 'Akshay' — still alive

} else {

  console.log('Object was garbage collected');

}

// FinalizationRegistry — run callback AFTER GC collects object

const registry = new FinalizationRegistry((label) => {

  console.log(`${label} was collected! Clean up side effects`);

});

let cache = { data: 'big data' };

registry.register(cache, 'cache object');

cache = null; // when GC collects -> callback fires automatically
```

WeakMap & WeakSet — Leak-Safe Collections

Intermediate -> Expert

```
// Regular Map PREVENTS garbage collection of keys

const map = new Map();

let user = { name: 'Akshay' };

map.set(user, 'some metadata');
```

```

user = null; // user object STILL can't be GC'd — Map holds it!

// WeakMap — keys held WEAKLY (don't prevent GC)

const weakMap = new WeakMap();

let user2 = { name: 'Rahul' };

weakMap.set(user2, 'metadata');

user2 = null; // user2 CAN be GC'd now — WeakMap won't prevent it!

// Real use — attaching metadata to DOM elements safely

const elementData = new WeakMap();

const btn = document.querySelector('#myBtn');

elementData.set(btn, { clickCount: 0 });

// When btn removed from DOM -> metadata auto-cleaned by GC

```

Feature	Map	WeakMap
Key types	Any value	Objects only
Prevents GC of keys?	Yes (strong refs)	No (weak refs)
Iterable?	Yes (forEach, for..of)	No
Has .size property?	Yes	No
Best for	General key-value storage	DOM metadata, caching

V8 Garbage Collector — How it Works

Expert

V8 divides the heap into two generations. Most objects die young (short-lived variables) so GC focuses there first for speed.

V8 HEAP

```
+-----+ +-----+
| YOUNG GENERATION | | OLD GENERATION |
| (Nursery) | | (Tenured) |
| | | |
| Newly created objects | | Survived 2+ GC cycles |
| GC runs very often | | GC runs rarely |
| Fast and cheap | | Slower, mark-sweep-compact|
+-----+ +-----+
```

Minor GC (Scavenge) -> cleans Young Gen, very fast

Major GC (Full) -> cleans Old Gen, pauses app briefly

SECTION 06 — DETECTING MEMORY LEAKS

How to Detect Leaks — Chrome DevTools

Intermediate

HEAP SNAPSHOT METHOD:

1. Open Chrome DevTools -> Memory tab
2. Take Heap Snapshot (baseline)
3. Perform the action that might leak (e.g. open/close modal)
4. Take another Heap Snapshot
5. Compare snapshots – look for objects that grew unexpectedly

PERFORMANCE TAB METHOD:

1. Start recording in Performance tab
2. Use your app normally for 30-60 seconds
3. Stop recording and inspect the memory graph

Healthy: /\ /\ /\ rises and falls (GC collecting)

Leaking: ////////// only rises, never falls

Memory Leak Prevention Checklist

```
// 1. Always declare variables with let/const

'use strict'; // catches accidental globals


// 2. Clear all timers when no longer needed

const id = setInterval(fn, 1000);

clearInterval(id); //


// 3. Remove event listeners on cleanup

element.removeEventListener('click', handler); //


// 4. Null heavy references when done

let bigData = loadHeavyData();

processData(bigData);

bigData = null; // allow GC


// 5. Use WeakMap for DOM element metadata

const meta = new WeakMap(); // auto-cleans


// 6. Extract only needed data before closure

const needed = bigArray[0]; // not the whole array

const fn = () => needed; // small closure


// 7. React — always return cleanup from useEffect

useEffect(() => {

  const sub = store.subscribe(handler);

  return () => sub.unsubscribe(); // cleanup!

}, []);
```

INTERVIEW Q & A

Top Interview Questions — Memory Leaks & GC

Q: What is a memory leak in JavaScript?

-> When code holds references to objects that are no longer needed, preventing the GC from freeing them. Heap grows continuously until the app slows down or crashes.

Q: What are the most common causes of memory leaks?

-> Accidental global variables, forgotten `setInterval/setTimeout`, event listeners not removed, closures holding large outer scopes, and detached DOM nodes still referenced in JS variables.

Q: How does the Garbage Collector know what to free?

-> Through the Mark and Sweep algorithm — starts from root (global scope), marks every reachable object, then sweeps (frees) everything not marked as reachable.

Q: What is the difference between WeakMap and Map regarding memory?

-> Map holds strong references to keys — even if you null your variable, Map prevents GC. WeakMap holds weak references — if no other reference to the key exists, GC can collect it automatically.

Q: What does WeakRef.deref() return?

-> The original object if it hasn't been GC'd yet, or undefined if it has. Always check the return value before using it.

Q: What are Minor GC and Major GC in V8?

-> Minor GC (Scavenge) cleans the Young Generation frequently and quickly. Major GC (Mark-Sweep-Compact) cleans the Old Generation less often but takes longer — it compacts memory too.

Golden Rules — Memory Safety

Reachable = kept alive by GC. Unreachable = freed by GC.

Leak = accidentally keeping a reference you no longer need.

WeakMap = safe metadata on objects/DOM without preventing GC.

Always clear intervals, remove listeners, null heavy refs.